

MINI PROJECT ON ADVANCED LIBRARIES IN PYTHON

```
import tkinter as tk

from tkinter import messagebox

import random

import json

import os

import winsound

import time


class TicTacToe:

    def __init__(self, root):

        self.root = root


# Professional centered window size

self.root.title("Tic Tac Toe")

window_width = 400

window_height = 520

screen_width = self.root.winfo_screenwidth()

screen_height = self.root.winfo_screenheight()

center_x = int((screen_width / 2) - (window_width / 2))

center_y = int((screen_height / 2) - (window_height / 2))

self.root.geometry(f'{window_width}x{window_height}+{center_x}+{center_y}')

self.root.resizable(False, False)


self.bg_color = "#f0f0f0"
```

```
self.x_color = "#ff5252"  
self.o_color = "#4caf50"  
self.win_color = "#ffff99"
```

```
self.current_player = "X"  
self.board = [""] * 9  
self.game_active = True
```

```
self.vs_ai = True  
self.player_symbol = "X"  
self.ai_symbol = "O"
```

```
self.score = {"Player": 0, "AI": 0, "Draws": 0}  
self.stats_file = "tic_tac_toe_stats.json"  
self.load_stats()
```

```
self.build_ui()
```

```
def build_ui(self):  
self.status_label = tk.Label(self.root, text="Player X's turn", font=("Helvetica", 14),  
bg=self.bg_color)  
self.status_label.pack(pady=10)
```

```
self.frame = tk.Frame(self.root, bg=self.bg_color)  
self.frame.pack()
```

```
self.cells = []  
for i in range(9):
```

```

        btn = tk.Canvas(self.frame, width=100, height=100, bg="white",
                        highlightthickness=1, highlightbackground="black")

        btn.grid(row=i//3, column=i%3)

        btn.bind("<Button-1>", lambda e, idx=i: self.cell_clicked(idx))

        self.cells.append(btn)

    self.btn_frame = tk.Frame(self.root)

    self.btn_frame.pack(pady=10)

    self.restart_btn = tk.Button(self.btn_frame, text="↺ Restart Game",
                                command=self.restart_game)

    self.restart_btn.grid(row=0, column=0, padx=5)

    self.symbol_btn = tk.Button(self.btn_frame, text="Play as O",
                                command=self.toggle_symbol)

    self.symbol_btn.grid(row=0, column=1, padx=5)

    self.ai_toggle_btn = tk.Button(self.btn_frame, text="Vs AI: On",
                                    command=self.toggle_ai)

    self.ai_toggle_btn.grid(row=0, column=2, padx=5)

    self.theme_toggle_btn = tk.Button(self.btn_frame, text="Switch Theme",
                                       command=self.toggle_theme)

    self.theme_toggle_btn.grid(row=0, column=3, padx=5)

    self.score_frame = tk.Frame(self.root, bg=self.bg_color)

    self.score_frame.pack(pady=5)

    self.score_label = tk.Label(self.score_frame, text=self.get_score_text(),
                                font=("Helvetica", 12), bg=self.bg_color)

```

```

self.score_label.grid(row=0, column=0, padx=5)

self.score_restart_btn = tk.Button(self.score_frame, text="↺ Reset Score",
                                   command=self.reset_score)

self.score_restart_btn.grid(row=0, column=1, padx=5)

self.timer_label = tk.Label(self.root, text="Time: 0s", font=("Helvetica", 12),
                             bg=self.bg_color)

self.timer_label.pack(pady=5)

self.start_time = time.time()

self.update_timer()

self.dark_mode = False

def toggle_ai(self):
    self.vs_ai = not self.vs_ai

self.ai_toggle_btn.config(text=f"Vs AI: {'On' if self.vs_ai else 'Off'}")

def toggle_symbol(self):
    self.player_symbol, self.ai_symbol = self.ai_symbol, self.player_symbol

    self.symbol_btn.config(text=f"Play as {self.player_symbol}")

    self.restart_game()

def toggle_theme(self):
    self.dark_mode = not self.dark_mode

    theme_color = "#000000" if self.dark_mode else "#f0f0f0"
    text_color = "#ffffff" if self.dark_mode else "#000000"

    self.root.config(bg=theme_color)

```

```

self.status_label.config(bg=theme_color, fg=text_color)
self.timer_label.config(bg=theme_color, fg=text_color)
    self.frame.config(bg=theme_color)
    self.btn_frame.config(bg=theme_color)
    self.score_frame.config(bg=theme_color)
self.score_label.config(bg=theme_color, fg=text_color)

    def update_timer(self):
        elapsed_time = int(time.time() - self.start_time)
        self.timer_label.config(text=f"Time: {elapsed_time}s")
        self.root.after(1000, self.update_timer)

    def cell_clicked(self, idx):
        if not self.game_active or self.board[idx] != "":
            return
        self.make_move(idx, self.current_player)
        self.play_sound("click")
        if self.check_game_end():
            return
        if self.vs_ai and self.current_player == self.ai_symbol:
            self.root.after(300, self.ai_move)

    def make_move(self, idx, symbol):
        self.board[idx] = symbol
        self.draw_symbol(idx, symbol)
        self.current_player = "O" if self.current_player == "X" else "X"
self.status_label.config(text=f"Player {self.current_player}'s turn")

```

```

def draw_symbol(self, idx, symbol):
    canvas = self.cells[idx]
    canvas.delete("all")
    if symbol == "X":
        canvas.create_line(20, 20, 80, 80, width=4, fill=self.x_color)
        canvas.create_line(80, 20, 20, 80, width=4, fill=self.x_color)
    elif symbol == "O":
        canvas.create_oval(20, 20, 80, 80, width=4, outline=self.o_color)

```

```

def check_winner(self):
    combos = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    for a,b,c in combos:
        if self.board[a] == self.board[b] == self.board[c] != "":
            for i in [a, b, c]:
                self.cells[i].configure(bg=self.win_color)
            return self.board[a]
    return None

```

```

def check_game_end(self):
    winner = self.check_winner()
    if winner:
        self.game_active = False
    winner_type = "Player" if winner == self.player_symbol else "AI"
    self.score[winner_type] += 1

```

```

        self.save_stats()

    self.status_label.config(text=f"{winner_type} wins!")

    self.score_label.config(text=self.get_score_text())

    messagebox.showinfo("Game Over", f"🏆 {winner_type} wins the game! 🏆")

    self.play_sound("win")

    return True

    elif "" not in self.board:

    self.game_active = False

    self.score["Draws"] += 1

    self.save_stats()

    self.status_label.config(text="It's a draw!")

    self.score_label.config(text=self.get_score_text())

    messagebox.showinfo("Game Over", " It's a Draw")

    self.play_sound("draw")

    return True

    return False


def restart_game(self):

    self.board = [""] * 9

    self.game_active = True

    self.current_player = "X"

    for cell in self.cells:

    cell.configure(bg="white")

    cell.delete("all")

    self.status_label.config(text=f"Player {self.current_player}'s turn")

    self.start_time = time.time()


def reset_score(self):

```

```

        self.score = {"Player": 0, "AI": 0, "Draws": 0}
        self.save_stats()

    self.score_label.config(text=self.get_score_text())

    def get_score_text(self):
    return f"Player: {self.score['Player']} | AI: {self.score['AI']} | Draws: {self.score['Draws']}"

    def save_stats(self):
    with open(self.stats_file, "w") as f:
        json.dump(self.score, f)

    def load_stats(self):
    if os.path.exists(self.stats_file):
    with open(self.stats_file, "r") as f:
        self.score = json.load(f)

    def ai_move(self):
    best_score = -float('inf')
    best_move = None
    for i in range(9):
    if self.board[i] == "":
    self.board[i] = self.ai_symbol
    score = self.minimax(0, False)
    self.board[i] = ""
    if score > best_score:
    best_score = score
    best_move = i
    if best_move is not None:

```



```
self.make_move(best_move, self.ai_symbol)
    self.check_game_end()
```

```
def minimax(self, depth, is_maximizing):
    winner = self.check_winner()
    if winner == self.ai_symbol:
        return 1
    elif winner == self.player_symbol:
        return -1
    elif "" not in self.board:
        return 0
```

```
        if is_maximizing:
            best = -float('inf')
            for i in range(9):
                if self.board[i] == "":
                    self.board[i] = self.ai_symbol
                    score = self.minimax(depth + 1, False)
                    self.board[i] = ""
                    best = max(score, best)
            return best
        else:
            best = float('inf')
            for i in range(9):
                if self.board[i] == "":
                    self.board[i] = self.player_symbol
                    score = self.minimax(depth + 1, True)
                    self.board[i] = ""
```

```
best = min(score, best)
```

```
return best
```

```
def play_sound(self, event):
```

```
    try:
```

```
        if event == "win":
```

```
            winsound.Beep(1000, 500)
```

```
        elif event == "draw":
```

```
            winsound.Beep(500, 500)
```

```
        elif event == "click":
```

```
            winsound.Beep(250, 100)
```

```
    except:
```

```
        pass
```

```
if __name__ == "__main__":
```

```
    root = tk.Tk()
```

```
    game = TicTacToe(root)
```

```
    root.mainloop()
```